PYTHON ACTIVITY-BASED LEARNING

Report on 10 Selected Case Studies

Prepared using basic Python concepts: variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, and tuple operations.

Student Name: _____

Roll Number: _____

Class / Section: _____

Subject: Python Programming

Date: _____

Note: The examples and test data in this report are illustrative. Review the code and replace the student details before submission.

Introduction

This report contains ten case studies selected from the supplied Python Activity-Based Learning list. Each case study includes a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion. The programs are kept simple so they can be understood and run by a beginner.

Selected Case Studies

12. Attendance Recovery Computation

13. ATM Transaction Sequence Simulation

14. Warehouse Inventory Reconciliation

17. Student Performance Trend Analysis

23. Student Timetable Conflict Detection

26. Nested Loop Examination Seating Generator

29. Password Policy Validation Engine

35. Text Compression Using Character Runs

39. Multi-Stage Examination Eligibility System

42. Student Marks Matrix Analysis

General note: The flowcharts are written as simple text flowcharts using Start, Input, Process, Decision, Output, and End. They can be redrawn with standard flowchart symbols if required by the instructor.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and find the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

2. Objectives

• Calculate attendance percentage using attended and conducted classes.

• Use a loop to find the minimum extra classes needed.

• Report the result clearly.

3. Assumptions

• The number of classes conducted is greater than zero.

• The student attends every future class counted by the program.

• The required attendance is between 0 and 100 percent.

4. Algorithm

1. Start.

2. Read classes conducted, classes attended, and target percentage.

3. Validate the inputs.

4. Calculate current attendance percentage.

5. While the projected percentage is below the target, add one conducted and one attended class.

6. Display the current percentage and extra classes needed.

7. Stop.

5. Flowchart

```
START → INPUT class counts and target → VALIDATE INPUT
   → CALCULATE CURRENT % → CHECK projected % >= target?
      No → ADD one attended class → CHECK again
      Yes → DISPLAY current % and extra classes → END
```

6. Pseudocode

```
BEGIN
INPUT conducted, attended, target
IF conducted <= 0 OR attended < 0 OR attended > conducted THEN
    PRINT 'Invalid input'
ELSE
    current = attended / conducted * 100
    extra = 0
    WHILE (attended + extra) / (conducted + extra) * 100 < target
        extra = extra + 1
    END WHILE
    PRINT current, extra
END IF
END
```

7. Python Code

```
conducted = int(input("Classes conducted: "))
attended = int(input("Classes attended: "))
target = float(input("Required attendance (%): "))

if conducted <= 0 or attended < 0 or attended > conducted or not 0 <= target <= 100:
    print("Invalid input")
else:
    current = attended / conducted * 100
    extra = 0

    while (attended + extra) / (conducted + extra) * 100 < target:
        extra += 1

    print(f"Current attendance: {current:.2f}%")
    print(f"Extra classes to attend: {extra}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	60	45
2	80	56
3	50	20

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Classes conducted: 80
Classes attended: 56
Required attendance (%): 75

>>> SAMPLE OUTPUT
Current attendance: 70.00%
Extra classes to attend: 16
```

10. Conclusion

The program calculates attendance and uses a loop to find the minimum extra classes required to meet the target.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Process a list of deposits and withdrawals, maintain the account balance, reject invalid transactions, and summarize the accepted transactions.

2. Objectives

• Update balance for deposits and withdrawals.

• Reject negative amounts and withdrawals beyond the balance.

• Count successful and rejected transactions.

3. Assumptions

• The opening balance cannot be negative.

• Transactions are entered as a type and amount.

• A withdrawal is accepted only when enough money is available.

4. Algorithm

1. Start with the opening balance.

2. Read the number of transactions.

3. For each transaction, read type and amount.

4. Accept a positive deposit or a valid withdrawal.

5. Reject invalid transactions and show the final summary.

6. Stop.

5. Flowchart

```
START → INPUT opening balance and count → LOOP over transactions
   → CHECK amount/type/balance → UPDATE balance or reject
   → DISPLAY final balance and counts → END
```

6. Pseudocode

```
BEGIN
INPUT balance, n
SET successful = 0, rejected = 0
REPEAT n times
    INPUT type, amount
    IF amount <= 0 THEN reject
    ELSE IF type = 'D' THEN balance = balance + amount
    ELSE IF type = 'W' AND amount <= balance THEN balance = balance - amount
    ELSE reject
    END IF
END REPEAT
PRINT balance, successful, rejected
END
```

7. Python Code

```
balance = float(input("Opening balance: "))
n = int(input("Number of transactions: "))
successful = 0
rejected = 0

if balance < 0 or n < 0:
    print("Invalid input")
else:
    for i in range(n):
        kind = input(f"Transaction {i + 1} (D for deposit, W for withdrawal): ").upper()
        amount = float(input("Amount: "))

        if amount <= 0:
            print("Rejected: amount must be positive")
            rejected += 1
        elif kind == "D":
            balance += amount
            successful += 1
            print("Deposit accepted")
```

```
    elif kind == "W" and amount <= balance:
        balance -= amount
        successful += 1
        print("Withdrawal accepted")
    else:
        print("Rejected: invalid type or insufficient balance")
        rejected += 1

print(f"Final balance: Rs. {balance:.2f}")
print(f"Successful transactions: {successful}")
print(f"Rejected transactions: {rejected}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	1000;D 500;W 200	Final Rs. 1300; success 2; rejected 0
2	100;W 150	Final Rs. 100; success 0; rejected 1
3	500;D -20	Final Rs. 500; success 0; rejected 1

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Opening balance: 1000
Number of transactions: 2
Transaction 1: D
Amount: 500
Transaction 2: W
Amount: 200

>>> SAMPLE OUTPUT
Deposit accepted
Withdrawal accepted
Final balance: Rs. 1300.00
Successful transactions: 2
Rejected transactions: 0
```

10. Conclusion

The simulation maintains the balance and gives a simple summary of accepted and rejected transactions.

Case Study 14: Warehouse Inventory Reconciliation

1. Problem Statement

Compare the expected closing stock with the actual closing stock for multiple products and report any difference and replenishment need.

2. Objectives

• Calculate expected stock from opening, received, and dispatched quantities.

• Compare expected and actual stock.

• Find products that need replenishment.

3. Assumptions

• Quantities are whole numbers and cannot be negative.

• Expected closing stock = opening + received - dispatched.

• The target stock is supplied for each product.

4. Algorithm

1. Start.

2. Read product details.

3. Calculate expected closing stock.

4. Compare it with actual stock and calculate the difference.

5. If stock is below target, calculate replenishment quantity.

6. Display each product's report.

5. Flowchart

```
START → READ product records → CALCULATE expected stock
   → COMPARE actual and expected → CALCULATE replenishment
   → DISPLAY stock report → NEXT product / END
```

6. Pseudocode

```
BEGIN
FOR each product
    INPUT name, opening, received, dispatched, actual, target
    expected = opening + received - dispatched
    difference = actual - expected
    shortage = MAX(0, target - actual)
    PRINT expected, difference, shortage
END FOR
END
```

7. Python Code

```
products = [
    ("Pen", 100, 40, 70, 65, 80),
    ("Notebook", 50, 30, 20, 60, 70),
    ("Pencil", 80, 10, 25, 65, 50)
]

print("Product | Expected | Actual | Difference | Replenish")
for name, opening, received, dispatched, actual, target in products:
    expected = opening + received - dispatched
    difference = actual - expected
    replenish = max(0, target - actual)
    print(f"{name:8} | {expected:8} | {actual:6} | {difference:10} | {replenish:9}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Pen: 100,40,70,65,80	Expected 70; difference -5; replenish 15
2	Notebook: 50,30,20,60,70	Expected 60; difference 0; replenish 10
3	Pencil: 80,10,25,65,50	Expected 65; difference 0; replenish 0

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Sample product records are entered in the products list in the program.

>>> SAMPLE OUTPUT
Product  Expected  Actual  Difference  Replenish
Pen      70        65      -5          15
Notebook 60        60      0           10
Pencil   65        65      0           0
```

10. Conclusion

The program identifies stock differences and calculates how many units are needed to reach the target stock.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Compare a student's marks across assessments, calculate improvement or decline, and identify subjects that remain below a minimum mark.

2. Objectives

• Store marks in a nested list.

• Calculate changes between assessments.

• Find subjects below the set minimum.

3. Assumptions

• Each mark is between 0 and 100.

• Each row contains marks for the same subjects in assessment order.

• The minimum acceptable mark is 40.

4. Algorithm

1. Store subject names and assessment marks.

2. For each subject, compare the first and last assessment.

3. Calculate the change.

4. Mark subjects whose final score is below 40.

5. Display the report.

5. Flowchart

```
START → READ marks matrix → FOR each subject calculate change
   → CLASSIFY trend → CHECK final mark < 40
   → DISPLAY subject report → END
```

6. Pseudocode

```
BEGIN
SET subjects and marks
FOR each subject index
    change = last_mark - first_mark
    IF change > 0 THEN status = 'Improved'
    ELSE IF change < 0 THEN status = 'Declined'
    ELSE status = 'No change'
    IF last_mark < 40 THEN PRINT 'Needs attention'
    PRINT subject, change, status
END FOR
END
```

7. Python Code

```
subjects = ["Python", "Maths", "English"]
marks = [
    [55, 62, 70],
    [68, 60, 58],
    [35, 38, 39]
]
minimum_mark = 40

print("Subject  First  Latest  Change  Trend  Note")
for i in range(len(subjects)):
    first = marks[i][0]
    latest = marks[i][-1]
    change = latest - first

    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "No change"

    note = "Needs attention" if latest < minimum_mark else "OK"
    print(f"{subjects[i]:8} {first:5} {latest:7} {change:7}  {trend:9} {note}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Python: 55 to 70	Change +15, Improved, OK
2	Maths: 68 to 58	Change -10, Declined, OK
3	English: 35 to 39	Change +4, Improved, Needs attention

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Marks are stored in the nested list in the program.

>>> SAMPLE OUTPUT
Subject  First  Latest  Change  Trend      Note
Python      55      70      15  Improved   OK
Maths       68      58     -10  Declined   OK
English     35      39       4  Improved   Needs attention
```

10. Conclusion

The program shows which subjects improved or declined and highlights scores that are still below the minimum.

Case Study 23: Student Timetable Conflict Detection

1. Problem Statement

Check a timetable represented by tuples and identify classes that use the same room at overlapping times.

2. Objectives

• Represent timetable entries using tuples.

• Compare class time intervals.

• Report room clashes.

3. Assumptions

• Times use 24-hour integer format, such as 9 for 09:00.

• Each class has a start time and end time on the same day.

• A clash exists when two classes overlap in the same room.

4. Algorithm

1. Store timetable entries as tuples.

2. Compare each entry with the entries after it.

3. Check if the room is the same and times overlap.

4. Print every conflict or report that none exists.

5. Flowchart

```
START → READ schedule tuples → COMPARE each pair
   → SAME ROOM and OVERLAPPING TIMES?
      Yes → DISPLAY conflict
      No → CHECK next pair → END
```

6. Pseudocode

```
BEGIN
FOR i from 0 to length(schedule)-1
    FOR j from i+1 to length(schedule)-1
        IF room_i = room_j AND start_i < end_j AND start_j < end_i
            PRINT conflict
        END IF
    END FOR
END FOR
END
```

7. Python Code

```
# Each tuple: (course, room, start_hour, end_hour)
schedule = [
    ("Python", "Lab-1", 9, 11),
    ("Maths", "Room-2", 10, 12),
    ("Web Design", "Lab-1", 10, 12),
    ("English", "Room-3", 12, 13)
]

found = False
for i in range(len(schedule)):
    course1, room1, start1, end1 = schedule[i]
    for j in range(i + 1, len(schedule)):
        course2, room2, start2, end2 = schedule[j]
        if room1 == room2 and start1 < end2 and start2 < end1:
            print(f"Conflict: {course1} and {course2} in {room1}")
            found = True

if not found:
    print("No room conflicts found")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Python 9-11 and Web Design 10-12 in Lab-1	Conflict found
2	Maths Room-2 10-12 and English Room-3 12-13	No conflict

Test	Input / Condition	Expected Result
3	Same room, classes 9-10 and 10-11	No conflict because they meet at the boundary

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Timetable entries are stored in the schedule list.

>>> SAMPLE OUTPUT
Conflict: Python and Web Design in Lab-1
```

10. Conclusion

The program uses tuples and nested loops to find classes that overlap in the same room.

Case Study 26: Nested Loop Examination Seating Generator

1. Problem Statement

Generate a classroom seating arrangement with row and seat numbers while keeping selected seats reserved.

2. Objectives

• Use nested loops to display rows and seats.

• Keep reserved seats unavailable.

• Display a simple seating plan.

3. Assumptions

• The classroom has a fixed number of rows and seats per row.

• Reserved seats are listed as (row, seat) pairs using 1-based numbering.

• All other seats are available.

4. Algorithm

1. Set the number of rows and seats per row.

2. Store reserved seat positions.

3. Use nested loops to visit each seat.

4. Print RES for reserved seats and seat numbers for available seats.

5. Flowchart

```
START → SET rows, seats and reserved positions → LOOP through rows
   → LOOP through seats → RESERVED?
      Yes → PRINT RES
      No → PRINT seat number
   → NEXT seat/row → END
```

6. Pseudocode

```
BEGIN
SET rows, seats_per_row, reserved
FOR row = 1 TO rows
    FOR seat = 1 TO seats_per_row
        IF (row, seat) in reserved THEN PRINT 'RES'
        ELSE PRINT seat
    END FOR
    PRINT newline
END FOR
END
```

7. Python Code

```
rows = 3
seats_per_row = 4
reserved = [(1, 3), (2, 2), (3, 4)]

for row in range(1, rows + 1):
    print(f"Row {row}: ", end="")
    for seat in range(1, seats_per_row + 1):
        if (row, seat) in reserved:
            print("RES", end="  ")
        else:
            print(f"S{seat}", end="   ")
    print()
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Reserved seat (1,3)	Row 1 shows RES at seat 3
2	Reserved seat (2,2)	Row 2 shows RES at seat 2
3	Reserved seat (3,4)	Row 3 shows RES at seat 4

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Rows, seats and reserved positions are set at the top of the program.

>>> SAMPLE OUTPUT
Row 1: S1   S2   RES  S4
Row 2: S1   RES  S3   S4
Row 3: S1   S2   S3   RES
```

10. Conclusion

The nested loops generate a clear seating plan and mark reserved positions.

Case Study 29: Password Policy Validation Engine

1. Problem Statement

Check sample passwords against basic rules and explain which rules are not satisfied.

2. Objectives

• Check password length.

• Check for uppercase, lowercase, digit, and special character.

• Report all failed rules.

3. Assumptions

• A valid password has at least 8 characters.

• It must contain uppercase, lowercase, digit, and a special character.

• This is an educational checker and not a complete security audit.

4. Algorithm

1. Read a sample password.

2. Create an empty list of errors.

3. Check each password rule.

4. Add a message for each failed rule.

5. Print whether the password passes and list any errors.

5. Flowchart

```
START → INPUT password → CHECK length and character rules
   → ADD each failed rule to list → LIST empty?
      Yes → PRINT passes
      No → PRINT failed rules → END
```

6. Pseudocode

```
BEGIN
INPUT password
SET errors = empty list
IF length < 8 THEN add length error
IF no uppercase THEN add uppercase error
IF no lowercase THEN add lowercase error
IF no digit THEN add digit error
IF no special character THEN add special error
IF errors empty THEN PRINT valid
ELSE PRINT errors
END
```

7. Python Code

```
password = input("Enter a sample password: ")
errors = []

if len(password) < 8:
    errors.append("Use at least 8 characters")
if not any(ch.isupper() for ch in password):
    errors.append("Add an uppercase letter")
if not any(ch.islower() for ch in password):
    errors.append("Add a lowercase letter")
if not any(ch.isdigit() for ch in password):
    errors.append("Add a digit")
if not any(not ch.isalnum() for ch in password):
    errors.append("Add a special character")

if errors:
    print("Password does not meet the rules:")
    for error in errors:
        print("-", error)
else:
    print("Password meets all basic rules")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Study@123	Passes all rules
2	hello	Length, uppercase, digit, special character errors
3	HELLO123	Lowercase and special character errors

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Enter a sample password: Study@123

>>> SAMPLE OUTPUT
Password meets all basic rules
```

10. Conclusion

The program checks several simple password rules and explains what is missing from an invalid example.

Case Study 35: Text Compression Using Character Runs

1. Problem Statement

Compress a string by counting consecutive repeated characters and compare the original and encoded lengths.

2. Objectives

• Group consecutive repeated characters.

• Create a simple run-length encoded string.

• Check whether encoding reduces the length.

3. Assumptions

• The input is treated as a case-sensitive string.

• Each run is represented by its character followed by its count.

• The program still displays the encoded result if it is longer.

4. Algorithm

1. Read a string.

2. If it is empty, report that it is empty.

3. Track the current character and run count.

4. Append each run to the encoded string.

5. Compare the two lengths and print the result.

5. Flowchart

```
START → INPUT text → EMPTY?
   Yes → PRINT empty string
   No → COUNT consecutive characters and build encoding
   → COMPARE lengths → DISPLAY result → END
```

6. Pseudocode

```
BEGIN
INPUT text
IF text is empty THEN PRINT 'Empty string'
ELSE
    SET encoded = empty string, count = 1
    FOR each next character
        IF next = current THEN count = count + 1
        ELSE append current and count; set current = next; count = 1
    END FOR
    append final current and count
    PRINT encoded and lengths
END IF
END
```

7. Python Code

```
text = input("Enter text: ")

if text == "":
    print("Empty string")
else:
    encoded = ""
    current = text[0]
    count = 1

    for ch in text[1:]:
        if ch == current:
            count += 1
        else:
            encoded += current + str(count)
            current = ch
            count = 1

    encoded += current + str(count)
    print("Encoded text:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))
```

```
if len(encoded) < len(text):
    print("Encoding reduces the length")
else:
    print("Encoding does not reduce the length")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	aaabbc	a3b2c1; encoded length 6
2	aaaaaa	a6; encoded length 2
3	abcd	a1b1c1d1; encoding longer

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Enter text: aaaaaa

>>> SAMPLE OUTPUT
Encoded text: a6
Original length: 6
Encoded length: 2
Encoding reduces the length
```

10. Conclusion

The program demonstrates basic run-length encoding and checks whether the encoded text is shorter.

Case Study 39: Multi-Stage Examination Eligibility System

1. Problem Statement

Decide whether a student can attend the final examination using attendance, laboratory completion, internal marks, and fee clearance.

2. Objectives

• Check four examination requirements.

• List every unmet requirement.

• Give a final eligible or not eligible decision.

3. Assumptions

• Minimum attendance is 75%.

• All laboratory work must be completed.

• Minimum internal marks are 40 out of 100.

• Fees must be cleared.

4. Algorithm

1. Read attendance, lab status, internal marks, and fee status.

2. Check each requirement independently.

3. Store every failed requirement in a list.

4. If the list is empty, show eligible; otherwise show not eligible and reasons.

5. Flowchart

```
START → INPUT attendance, lab, marks and fee status
   → CHECK each requirement → COLLECT all failed conditions
   → ANY reasons?
      No → PRINT eligible
      Yes → PRINT not eligible and reasons → END
```

6. Pseudocode

```
BEGIN
INPUT attendance, lab_done, internal_marks, fees_paid
SET reasons = empty list
IF attendance < 75 THEN add attendance reason
IF lab_done is false THEN add lab reason
IF internal_marks < 40 THEN add internal marks reason
IF fees_paid is false THEN add fee reason
IF reasons empty THEN PRINT eligible
ELSE PRINT not eligible and reasons
END
```

7. Python Code

```
attendance = float(input("Attendance percentage: "))
lab_done = input("All labs completed? (yes/no): ").lower()
internal_marks = float(input("Internal marks out of 100: "))
fees_paid = input("Fees cleared? (yes/no): ").lower()

reasons = []

if attendance < 75:
    reasons.append("Attendance is below 75%")
if lab_done != "yes":
    reasons.append("Laboratory work is incomplete")
if internal_marks < 40:
    reasons.append("Internal marks are below 40")
if fees_paid != "yes":
    reasons.append("Fees are not cleared")

if not reasons:
    print("Result: Eligible for final examination")
else:
    print("Result: Not eligible for final examination")
    print("Requirements not met:")
    for reason in reasons:
```

```
        print("-", reason)
```

8. Test Cases

Test	Input / Condition	Expected Result
1	80, yes, 65, yes	Eligible
2	70, yes, 65, yes	Attendance requirement not met
3	70, no, 35, no	Three unmet requirements

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Attendance percentage: 70
All labs completed? (yes/no): no
Internal marks out of 100: 35
Fees cleared? (yes/no): no

>>> SAMPLE OUTPUT
Result: Not eligible for final examination
Requirements not met:
- Attendance is below 75%
- Laboratory work is incomplete
- Internal marks are below 40
- Fees are not cleared
```

10. Conclusion

The program checks all four conditions and reports every requirement that the student has not met.

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Analyse marks stored in a nested list, calculate each student's total and average, calculate subject averages, and identify the highest-scoring student.

2. Objectives

• Use a nested list to store marks.

• Calculate row-wise and column-wise statistics.

• Find the top performer and low-average subjects.

3. Assumptions

• Each mark is out of 100.

• Every student has marks for the same subjects.

• The sample has three students and three subjects.

4. Algorithm

1. Store student names, subject names, and marks.

2. Calculate each student's total and average.

3. Calculate each subject's average.

4. Find the student with the highest total.

5. Report subject averages and any subject below 50.

5. Flowchart

```
START → READ marks matrix → CALCULATE row totals/averages
   → CALCULATE subject averages → FIND highest total
   → DISPLAY summary → END
```

6. Pseudocode

```
BEGIN
SET students, subjects, marks
FOR each student row
    total = SUM(row); average = total / number_of_subjects
    PRINT total, average
END FOR
FOR each subject column
    average = SUM(column marks) / number_of_students
    PRINT average
END FOR
FIND student with maximum total
END
```

7. Python Code

```
students = ["Anu", "Bala", "Charan"]
subjects = ["Python", "Maths", "English"]
marks = [
    [85, 78, 90],
    [72, 88, 75],
    [60, 65, 70]
]

totals = []
print("Student results:")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(f"{students[i]}: Total = {total}, Average = {average:.2f}")

print("\nSubject averages:")
for j in range(len(subjects)):
    subject_total = 0
    for i in range(len(students)):
        subject_total += marks[i][j]
    average = subject_total / len(students)
```

```
    print(f"{subjects[j]}: {average:.2f}")

top_index = totals.index(max(totals))
print(f"Top performer: {students[top_index]} ({totals[top_index]} marks)")

for j in range(len(subjects)):
    subject_average = sum(marks[i][j] for i in range(len(students))) / len(students)
    if subject_average < 50:
        print(f"Low average subject: {subjects[j]}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Anu: 85,78,90	Total 253; average 84.33
2	Bala: 72,88,75	Total 235; average 78.33
3	Charan: 60,65,70	Total 195; average 65.00

9. Sample Run / Output Screenshot

The following is a text capture of the sample console output. Run the program in Python IDLE, VS Code, or another Python editor and take a screenshot of the console for your final submission if your instructor requires an actual screenshot.

```
>>> SAMPLE INPUT
Sample marks are stored in the nested list in the program.

>>> SAMPLE OUTPUT
Student results:
Anu: Total = 253, Average = 84.33
Bala: Total = 235, Average = 78.33
Charan: Total = 195, Average = 65.00

Subject averages:
Python: 72.33
Maths: 77.00
English: 78.33
Top performer: Anu (253 marks)
```

10. Conclusion

The program demonstrates nested-list processing, student averages, subject averages, and top-performer identification.